

Layer 1 — Version 4: `revalidate` → ISR

Now we get to **ISR**, and this is where the previous pieces start connecting beautifully.

We'll use:

```
export default async function ProductsPage() {
  const response = await fetch(
    "https://api.example.com/products",
    {
      next: {
        revalidate: 60
      }
    }
  )

  const products = await response.json()

  return (
    <div>
      <h1>Products</h1>

      {products.map((item) => (
        <div key={item.id}>
          {item.name}
        </div>
      ))}
    </div>
  )
}
```

The important part is:

```
next: {
  revalidate: 60
}
```

Meaning, at a high level:

This fetched data can be reused, but it should be revalidated after 60 seconds.

1 First: what gets cached?

We potentially have **two caches** again:

API
|
▼
Data Cache
|



▼
Rendering
|
▼
Full Route Cache

But now both are connected to revalidation.

Think:

Data Cache
↓
"How long can I reuse this fetched data?"

Full Route Cache
↓
"How long can I reuse this rendered route?"

For this example, the route can be statically rendered and regenerated based on the data's revalidation.

2 Imagine the first generation happens at 10:00

At 10:00:

/products

needs to be generated.

Next.js executes:

```
fetch(url, {  
  next: {  
    revalidate: 60  
  }  
})
```

Suppose the API says:

```
[  
  { "id": 1, "name": "iPhone" },
```

```
{ "id": 2, "name": "MacBook" }  
]
```

So:

```
10:00  
|  
▼  
fetch()  
|  
▼  
API  
|  
▼  
Data Cache  
|  
▼  
products  
|  
▼  
React rendering  
|  
▼  
HTML + RSC Payload  
|  
▼  
Full Route Cache
```



Now both caches have information.

3 User arrives at 10:10

Only **10 minutes** have passed.

The user requests:

```
GET /products
```



Next.js can reuse the rendered result:

```
10:10  
|  
▼  
Full Route Cache  
|  
| HIT  
▼
```



```
HTML + RSC
|
▼
HTTP Response
|
▼
Browser
```

We don't need to:

- ❌ call API
- ❌ execute Page()
- ❌ run map()
- ❌ regenerate HTML



The previously generated result can be reused.

4 User arrives at 10:50

Same thing.

```
10:50
|
▼
Full Route Cache
|
| still reusable
▼
cached rendered result
|
▼
Browser
```



Easy. 😎

5 Now we reach 11:00

60 minutes? No.

Our revalidate is:

60 seconds



So actually let's use the correct timeline:

10:00 → generated
10:01 → revalidation threshold



At 10:00:

Fresh



At 10:01:

Stale / eligible for revalidation



At 10:10:

Already stale



But here's the important ISR behavior.

6 What happens when a request arrives after the revalidation period?

Suppose:

10:05



User requests `/products`.

The cached result is now stale.

A simplified mental model is:

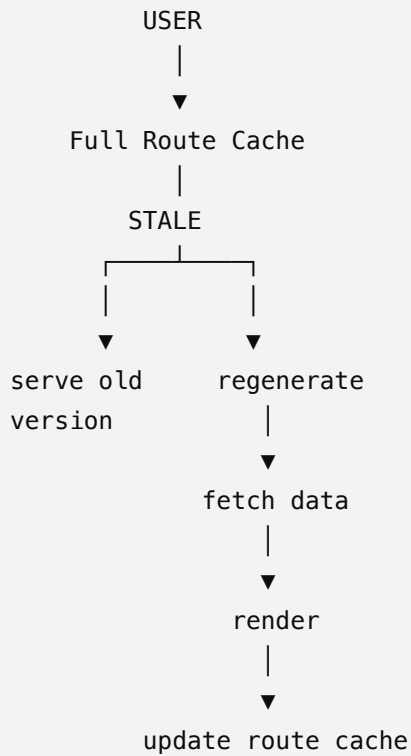
```
Request
  ↓
Full Route Cache
  ↓
stale
  ↓
serve existing result
  +
trigger regeneration
```



This is the famous **stale-while-revalidate-style** ISR behavior.

The user may receive the previous cached version while Next.js regenerates the page.

Conceptually:



🔥 That's the key ISR idea.

7 Suppose the API has changed

At 10:05 the API now says:

```
[
  { "id": 1, "name": "iPhone" },
  { "id": 2, "name": "MacBook" },
  { "id": 3, "name": "iPad" }
]
```

The regeneration process gets the new data.

Old:
iPhone
MacBook

New:
iPhone
MacBook
iPad

Next.js renders again:

```
New API data
  ↓
products
  ↓
map()
  ↓
new HTML
  +
new RSC Payload
  ↓
Full Route Cache updated
```



Now subsequent requests get:

```
iPhone
MacBook
iPad
```



8 Here's the really important relationship

You might now ask:

"Does `revalidate: 60` mean both Data Cache and Full Route Cache expire after exactly 60 seconds?"

Don't memorize it that way.

The precise relationship depends on **which cache is being configured and how the route's dependencies determine the route's revalidation.**

For our Layer-1 mental model, the important relationship is:

```
fetch revalidate
  ↓
Data Cache can be revalidated
  ↓
route depends on that data
  ↓
rendered route can be regenerated
  ↓
Full Route Cache updated
```



And Next.js uses the **lowest revalidation interval among cached data dependencies** to determine the route's revalidation frequency in the traditional model.

For example:

```
fetch(A, {  
  next: { revalidate: 3600 }  
})  
  
fetch(B, {  
  next: { revalidate: 60 }  
})
```

Then conceptually:

```
A → 1 hour  
B → 1 minute  
  
Route → 1 minute
```



Because the page depends on the data that needs refreshing sooner.

9 Compare our three versions

Now we have a very useful table.

Version 1 — No data

```
return <h1>Hello</h1>
```

```
Static  
↓  
Full Route Cache
```



Version 2 — Cacheable `fetch()`

```
fetch(url)
```

fetch
↓
Data Cache
↓
static rendering
↓
Full Route Cache



Version 3 — no-store

```
fetch(url, {  
  cache: "no-store"  
})
```

fetch
↓
✗ Data Cache
↓
request-time rendering
↓
✗ reusable Full Route Cache



Version 4 — revalidate

```
fetch(url, {  
  next: {  
    revalidate: 60  
  }  
})
```

fetch
↓
Data Cache
↓
static rendering
↓
Full Route Cache
↓
revalidation



```
↓  
regenerate  
↓  
update caches
```

That's **ISR** in the traditional mental model.

🔥 10 One subtle but important correction

Earlier I used a timeline like:

```
10:00  
10:30  
11:00  
11:01
```



with `revalidate: 60`.

That was just illustrative, but for **60 seconds**, the correct threshold is one minute.

More importantly, don't think:

"At exactly 60 seconds, a background timer wakes up and renders the page."

That's **not the right mental model**.

Think:

```
Page generated  
↓  
becomes stale after revalidation period  
↓  
a subsequent request provides the opportunity  
↓  
Next.js revalidates/regenerates
```

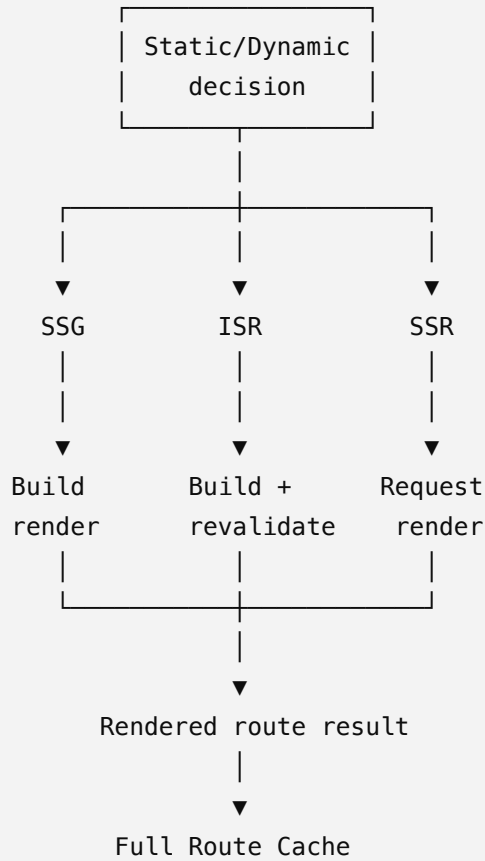


The exact regeneration behavior has additional nuances, but this is the mental model you want first.

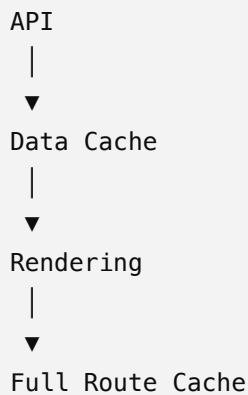
🧠 The entire Layer-1 rendering picture so far

```
ROUTE  
|  
▼
```





And underneath the data:



🎯 Next: `generateStaticParams()`

Now we've covered:

- ✓ Static page
- ✓ cacheable fetch
- ✓ no-store
- ✓ SSG
- ✓ SSR
- ✓ ISR

The next piece is:

```
export async function generateStaticParams() {  
  return [  
    { id: "1" },  
    { id: "2" },  
    { id: "3" }  
  ]  
}
```

with:

app/products/[id]/page.tsx



This will answer a very common interview question:

"How can a dynamic route like `/products/[id]` still be statically generated?"

And we'll trace:

/products/1
/products/2
/products/3
/products/999



and see **which ones exist ahead of time**, what happens for an unknown ID, and how `generateStaticParams()` interacts with the Full Route Cache.



ok

Layer 1 — Version 5: `generateStaticParams()`

Now we're going to solve a very common confusion:

"If my route contains `[id]`, how can Next.js possibly statically render it? The ID isn't known until the user requests the URL!"

Exactly. 🤔 That's what `generateStaticParams()` is for.

1 Start with a dynamic route

Suppose our project is:

```
app/  
└─ products/  
    └─ [id]/  
        └─ page.tsx
```



This creates URLs like:

```
/products/1  
/products/2  
/products/3  
/products/999
```



And our component:

```
<\/> TypeScript  
  
export default async function ProductPage({  
  params  
}) {  
  const { id } = await params  
  
  const response = await fetch(  
    `https://api.example.com/products/${id}`  
  )  
  
  const product = await response.json()  
  
  return (  
    <div>  
      <h1>{product.name}<\/h1>  
      <p>₹{product.price}<\/p>  
    <\/div>  
  )  
}
```



Notice:

```
[id]
```



is dynamic.

So how could Next.js know what pages to generate?

2 Enter `generateStaticParams()`

We can tell Next.js:

```
</> TypeScript

export async function generateStaticParams() {
  return [
    { id: "1" },
    { id: "2" },
    { id: "3" }
  ]
}
```

Now Next.js knows:

Generate:

```
/products/1
/products/2
/products/3
```

ahead of time.

3 Think of `[id]` as a template

This is a very useful mental model.

Your file:

```
products/[id]/page.tsx
```

is basically a **template**:

```
/products/{id}
```

And `generateStaticParams()` gives Next.js concrete values:

```
id = 1
id = 2
id = 3
```

So:

```
Template
/products/[id]
|
```

| generateStaticParams()



id = 1	→ /products/1
id = 2	→ /products/2
id = 3	→ /products/3

Next.js can then render each one.

4 What happens during build?

Suppose:

`</> TypeScript`

```
export async function generateStaticParams() {  
  return [  
    { id: "1" },  
    { id: "2" },  
    { id: "3" }  
  ]  
}
```

During build, conceptually:

next build



generateStaticParams()



[1, 2, 3]



/products/1



render



HTML+RSC



/products/2



render



HTML+RSC



/products/3



render



HTML+RSC



Full Route Cache

So the Full Route Cache might conceptually contain:

```
/products/1 → rendered result  
/products/2 → rendered result  
/products/3 → rendered result
```



🔥 That's the key.

5 Notice what happened to `params`

For `/products/1`:

```
</> TypeScript  
  
const { id } = await params
```



gives:

```
id = "1"
```



Then:

```
</> TypeScript  
  
fetch(`/products/${id}`)
```



becomes conceptually:

```
fetch("/products/1")
```



For `/products/2`:

```
id = "2"
```



and:

```
fetch("/products/2")
```



So Next.js can render each concrete page during the build.

6 What does the Full Route Cache contain?

Suppose product 1 is:

```
</> JSON

{
  "id": 1,
  "name": "iPhone",
  "price": 80000
}
```

The rendered result for `/products/1` could conceptually be:

```
</> HTML

<h1>iPhone</h1>
<p>₹80000</p>
```

And `/products/2` could be:

```
</> HTML

<h1>MacBook</h1>
<p>₹120000</p>
```

So think:

```
Full Route Cache

/products/1
  ↓
HTML + RSC Payload for iPhone

/products/2
  ↓
HTML + RSC Payload for MacBook

/products/3
  ↓
HTML + RSC Payload for AirPods
```

Again:

not the component code.

The component is the template used to produce each result.

7 Now the interesting question: `/products/999`

Suppose:

```
generateStaticParams()
```



only returned:

```
1  
2  
3
```



But a user requests:

```
/products/999
```



What happens?

This depends on the route's dynamic/static configuration and the behavior you're using, so let's introduce the important option:

```
</> TypeScript
```



```
export const dynamicParams = false
```

Now:

```
</> TypeScript
```



```
export async function generateStaticParams() {  
  return [  
    { id: "1" },  
    { id: "2" },  
    { id: "3" }  
  ]  
}  
  
export const dynamicParams = false
```

This means:

Only the parameters returned by `generateStaticParams()` are valid for this statically generated route.

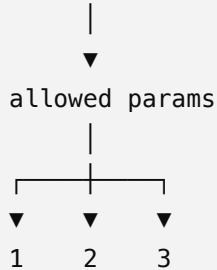
Therefore:

```
/products/1 → ✓  
/products/2 → ✓  
/products/3 → ✓  
/products/999 → ✗
```



Conceptually:

generateStaticParams



999 → not generated → 404

8 What if `dynamicParams` is allowed?

Now don't set:

`</> TypeScript`

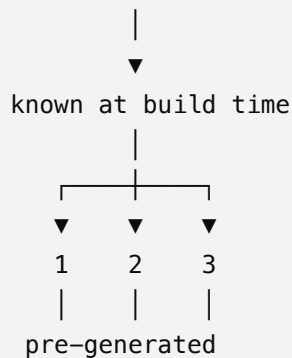


```
dynamicParams = false
```

Then a parameter that wasn't returned can potentially be generated when requested, depending on the route's rendering/caching configuration.

So:

generateStaticParams()



But:

```
/products/999
```



wasn't known during the build.

A request can cause Next.js to handle that route dynamically and potentially cache the resulting route according to the applicable caching rules.

The important mental distinction is:

`generateStaticParams()` tells Next.js which parameter values it should know about ahead of time. It does not necessarily mean those are the only values that can ever exist.

9 This is why `[id]` doesn't automatically mean SSR

This is a **very important interview point**.

You might have:

```
/products/[id]
```



and think:

```
[id]
  ↓
unknown
  ↓
SSR
```



✗ Not necessarily.

You can have:

```
/products/[id]
      +
generateStaticParams()
      ↓
known IDs
      ↓
pre-rendered
      ↓
Full Route Cache
```



So:

Dynamic route ≠ dynamically rendered route.

This distinction is HUGE.

"Dynamic route" here means the **URL has a dynamic segment**.

"Dynamic rendering" means the **page is rendered at request time**.

They're two different uses of the word **dynamic**.

🔥 100 This distinction will save you in interviews

Dynamic route

```
/products/[id]
```



means:

The URL has a variable segment.

Example:

```
/products/1  
/products/2  
/products/999
```



Dynamic rendering

means:

The route's UI needs to be generated at request time.

For example, traditionally:

```
</> TypeScript  
  
cookies()
```



or:

```
</> TypeScript  
  
fetch(url, {  
  cache: "no-store"  
})
```



can introduce request-time rendering.

So:

Dynamic ROUTE
≠
Dynamic RENDERING



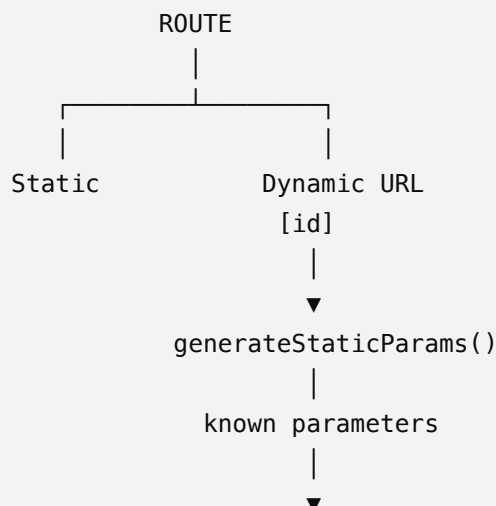
🔥 Memorize that distinction.

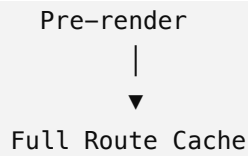
1 1 Let's compare everything we've learned

Example	Route type	Rendering
/	Static route	Static
/products	Static route	Static if dependencies allow
/products/[id]	Dynamic route	Can still be static
[id] + generateStaticParams()	Dynamic route	Pre-render known params
fetch(..., { cache: "no-store" })	Any route	Traditionally dynamic rendering
cookies()	Any route	Traditionally dynamic rendering

🧠 Full mental model

Now our Layer-1 pipeline looks like:





Then independently, rendering can be influenced by:

```
fetch()
cookies()
headers()
searchParams
no-store
revalidate
```



which we'll use to determine:

```
STATIC
OR
DYNAMIC
```

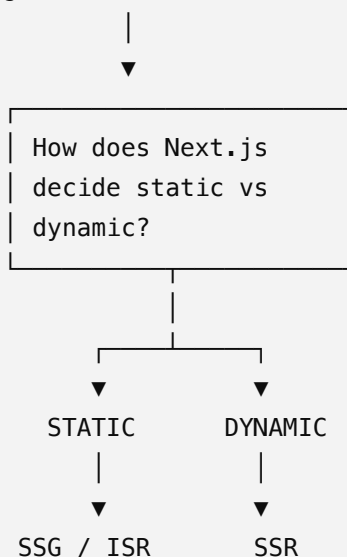


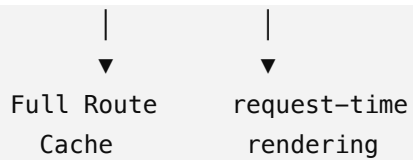
We've now reached the real decision tree

We've learned the individual pieces.

Now we're ready for the **thing you originally wanted to understand**:

```
fetch()
cookies()
headers()
searchParams
cache: "no-store"
revalidate
generateStaticParams()
```





Next lesson should be the decision tree itself — we'll take one route and add `fetch`, `no-store`, `cookies`, `headers`, and `searchParams` **one at a time**, and after each addition we'll ask:

"Did Next.js just change this route from static → dynamic? Why?"

That's where all the individual rules finally become one coherent system. 🧠🔥









Ahhh, yes — I see the exact confusion now. 😊 You're asking:

"If `/products` is cached and doesn't depend on the cookie, and I change a cookie in the browser, how does the server ever learn that the cookie changed?"

The answer is: **it learns it on the next request that actually reaches the server.** And if that request is satisfied entirely by a CDN cache, the origin server may **not need to know at all.**

Let's separate these cases.

1. Cookie changes in the browser

Suppose:

Browser

theme = light



Server Action changes it:

```
cookies().set("theme", "dark")
```



Response:

```
</> http
```

```
Set-Cookie: theme=dark
```



Browser now has:

Browser

theme = dark



At this point:

Server: ❌ hasn't received "theme=dark" as a new request

Browser: ✅ knows theme=dark



2. Then user requests `/products`

Browser makes:

```
</> http
```

```
GET /products
```

```
Cookie: theme=dark
```



Now there are two possibilities.

Case A — CDN handles it

Browser

|

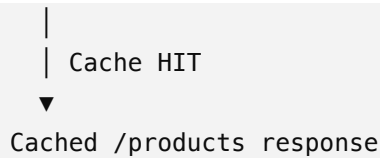
| GET /products

| Cookie: theme=dark



CDN





The request **doesn't reach Next.js origin**.

Therefore:

Next.js server
✗ doesn't learn about this request
✗ doesn't need to



The CDN has already answered the request.

And that's perfectly fine if `/products` doesn't care about `theme`.

3. So when does Next.js actually learn the new cookie?

When there is a request that **reaches Next.js**.

For example:



Now Next.js can see:

theme=dark



So:

Cookie changed
↓
Browser stores new cookie
↓



Future HTTP request
↓
Cookie automatically attached
↓
If request reaches server
↓
Server can read new cookie

4. But here's the important realization 💡

You don't actually need to "tell the server that the cookie changed" as a separate operation.

The cookie itself is the state.

The browser effectively says:

"From now on, whenever I make an applicable request, I'll send `theme=dark`."

So imagine:

10:00



Browser:
theme=light

Server Action:

Set-Cookie: theme=dark



Browser:

theme=dark



Then:

10:05



GET /profile
Cookie: theme=dark

Server sees it.

There was no separate:

UPDATE_COOKIE /api/notify-server



required.

5. What if `/products` is completely cached?

Suppose:

`/products`



doesn't care about cookies.

CDN has:

`/products → Product HTML`



User requests it:

Browser
Cookie: theme=dark
↓
CDN
↓
Cached Product HTML



Next.js doesn't learn about the cookie.

And that's okay.

Because the page doesn't use it.

It's exactly like sending someone a letter that says:

"Here is my favorite color: dark"



but the person only needs to answer:

"What products are available?"



They don't need to process your favorite color. 😊

6. Now compare with `/profile`

Suppose `/profile` says:

`</> TypeScript`

```
const cookieStore = await cookies()
const theme = cookieStore.get("theme")?.value
```



Now the cookie matters.

Browser:

`</> http`

```
GET /profile
Cookie: theme=dark
```



If CDN cannot safely serve a generic cached response:

Browser

↓

CDN

↓

Next.js

↓

`cookies()`

↓

`theme = dark`

↓

`render`



Now Next.js **does** learn about it.

7. There's one more scenario: you **WANT** the server to know immediately

Suppose you have some client-side cookie that you change using JavaScript:

`</> JavaScript`

```
document.cookie = "theme=dark"
```



The server isn't immediately notified.

If you specifically need the server to know immediately, you make a request:

```
</> JavaScript
```

```
document.cookie = "theme=dark"

await fetch("/api/theme", {
  method: "POST"
})
```

Now:

```
Browser
|
| Cookie already changed
| Cookie: theme=dark
▼
POST /api/theme
▼
Next.js
|
▼
Server knows theme=dark
```

But normally you don't need this extra request unless your application has a reason to synchronize server-side state.



This is the BIG distinction

There are actually **three events**:

Event 1 — Cookie changes

```
Server → Set-Cookie → Browser
```

or JavaScript changes it.

Event 2 — Browser makes a request

```
Browser → HTTP Request + Cookie → network
```

Event 3 — Somebody processes that request

Could be:

```
Browser
↓
```

CDN
↓
response

or

Browser
↓
CDN
↓
Next.js

If CDN answers it:

Next.js doesn't see it.

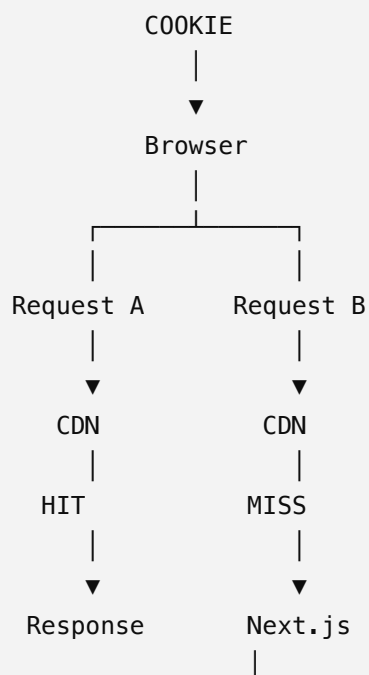
If CDN forwards it:

Next.js sees the cookie.

🔥 The sentence that should make this click

A cookie is not a live connection between the browser and server. It's client-side state that gets included in applicable HTTP requests.

So:



▼
cookies()

In **Request A**, Next.js never knows the cookie existed.

In **Request B**, Next.js sees the cookie.

And that's completely normal. 👍

The next thing that will make this whole topic click is understanding **what exactly a CDN cache key is and whether cookies are part of that key**—because that determines whether two users with different cookies can receive the same cached response.



ok

Perfect 😊 Let's lock this piece in.

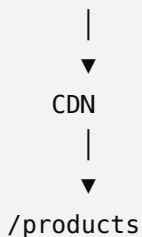
🧠 Next: CDN Cache Key + Cookies

The question we'll answer is:

If two users request `/products` but have different cookies, how does the CDN know whether it can give them the same cached response?

We'll trace:

User A
Cookie: theme=dark



User B
Cookie: theme=light



▼
/products

Then we'll understand:

1. What a cache key actually is
2. Why `/products` normally maps to one cached response
3. How `Vary` works
4. Whether cookies can be part of cache variation
5. Why personalized pages generally shouldn't be blindly CDN-cached
6. How this differs from **Next.js Data Cache / Full Route Cache / Router Cache**
7. Finally, we'll return to our **Layer 1 static vs dynamic exercise**. 🚀



ok

Alright 😊 let's do it. This is one of those concepts that makes **CDN caching + cookies** suddenly click.

🌐 **CDN Cache Key — What actually identifies a cached response?**

Imagine the CDN receives:

```
</> http
GET /products
```



The CDN needs to answer:

"Do I already have a response for this request?"

It uses something called a **cache key**.

A simplified cache key might be:

```
https://example.com/products
```



So:

```
Request
GET /products
```



↓
Cache Key
example.com/products
↓
CDN Cache
↓
Cached Response

1. Two users request the same URL

Suppose:

User A

GET /products
Cookie: theme=dark



User B

GET /products
Cookie: theme=light

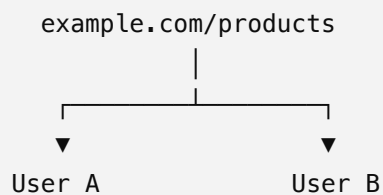


If the CDN's cache key is simply:

example.com/products



then both requests map to:



So the CDN could potentially return the **same cached response** to both.

That's perfectly fine **if the response doesn't depend on the cookie**.

2. Example where this is perfectly safe

Your `/products` page:

</> TypeScript



```
export default async function ProductsPage() {  
  const products = await getProducts()  
  
  return (  
    <ul>  
      {products.map(product => (  
        <li key={product.id}>{product.name}</li>  
      ))}  
    </ul>  
  )  
}
```

The page doesn't care about:

theme
session
language
user ID



So:

User A
Cookie: theme=dark
|
▼
/products
|
▼
CDN
|
▼
Cached products response



User B:

User B
Cookie: theme=light
|
▼
/products
|
▼
CDN
|
▼
SAME cached products response



That's fine.

Because:

Cookie —X→ Products HTML



The cookie doesn't influence the response.

3. Now let's make the cookie matter

Suppose:

`</>` TypeScript



```
export default async function ProductsPage() {  
  const cookieStore = await cookies()  
  
  const theme = cookieStore.get("theme")?.value  
  
  return (  
    <div className={theme}>  
      Products  
    </div>  
  )  
}
```

Now:

Cookie

↓

theme

↓

HTML



User A:

theme=dark

↓

<div class="dark">



User B:

theme=light

↓

<div class="light">



So one cached response cannot represent both.

4. Here's the problem with a naive CDN

Suppose CDN already has:

Cache:

/products

↓

<div class="dark">



User B arrives:

</> http

GET /products

Cookie: theme=light



If CDN says:

"Oh, /products is cached! I'll return it."

then:

User B

theme=light

↓

CDN

↓

<div class="dark">



💥 Wrong response.

That's a **cache correctness problem**.

5. So how does a CDN handle this?

There are several mechanisms in real systems.

One important HTTP mechanism is:

```
</> http
```



```
Vary
```

For example:

```
</> http
```



```
Vary: Accept-Language
```

This tells caches:

"The response varies based on this request header."

So conceptually:

```
/products + language=en
```

```
/products + language=fr
```



can be treated as different variants.

6. What about cookies?

Here's where things get more nuanced.

You *can* design caching behavior around cookies, but you generally don't want to casually make:

```
Cache Key =
```

```
URL + entire Cookie header
```



because imagine:

```
Cookie:
```

```
session=abc123;
```

```
theme=dark;
```

```
cart=92837;
```

```
...
```



Now every user's session could produce a completely different cache entry.

You'd destroy much of the benefit of shared caching.

You could theoretically get:

```
/products + userA cookie  
/products + userB cookie  
/products + userC cookie  
/products + userD cookie  
...
```



That's basically **personalized caching**.

7. This is why personalized pages are tricky

Imagine:

```
</> TypeScript  
  
const session = cookies().get("session")  
  
return <h1>Welcome, {sessionUser.name}</h1>
```



User A:

Welcome Gopi



User B:

Welcome Rahul



You absolutely cannot have:

```
/products  
  ↓  
Welcome Gopi
```



as one generic shared CDN response.

Otherwise:

```
User B  
  ↓  
CDN  
  ↓  
"Welcome Gopi"
```



That's obviously terrible.

So systems generally avoid treating such personalized responses as ordinary shared public cache entries, or explicitly vary/cache them according to an appropriate strategy.

8. Now connect this to Next.js

This is why our earlier statement was:

```
cookies()
  ↓
request-dependent
  ↓
dynamic rendering
```



Under the **traditional Layer 1 model**, Next.js doesn't say:

"I'll just put this personalized page in the generic Full Route Cache."

Instead, the route is treated as request-dependent.

Conceptually:

```
Request
  ↓
Next.js
  ↓
cookies()
  ↓
render THIS request
  ↓
response
```



9. But here's a very important distinction

Don't think:

"If a request contains a cookie, CDN can never cache it."

That's too simplistic.

The correct idea is:

The presence of a cookie doesn't automatically make every response uncacheable. What matters is whether the response is safe to reuse for another

request.

For example:

Cookie: theme=dark



but `/products` doesn't use theme.

The response could still potentially be shared.

Whereas:

Cookie: userId=123



and the response says:

Welcome Gopi



That's personalized.

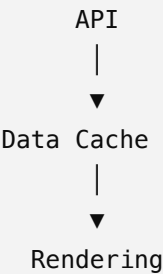
Different situation.

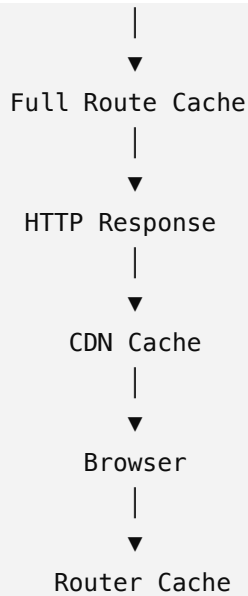
10. Now our three cache layers become clearer

Let's put them side-by-side.

Layer	What it caches	Where
Data Cache	Fetches data	Next.js/server infrastructure
Full Route Cache	Rendered route result	Next.js/server infrastructure
CDN/HTTP cache	HTTP responses	CDN/edge/intermediary
Router Cache	RSC payload	Browser/client

So:





And importantly, **these aren't duplicate copies of exactly the same thing for the same purpose.**

11. One killer example 🔥

Imagine your page:

`</>` TypeScript

```
export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get("theme")?.value

  const response = await fetch(
    "https://api.example.com/products",
    {
      next: { revalidate: 60 }
    }
  )

  const products = await response.json()

  return (
    <div className={theme}>
      ...
    </div>
  )
}
```

Now there are **two different kinds of information**:

Cookie

theme=dark



Request-specific.

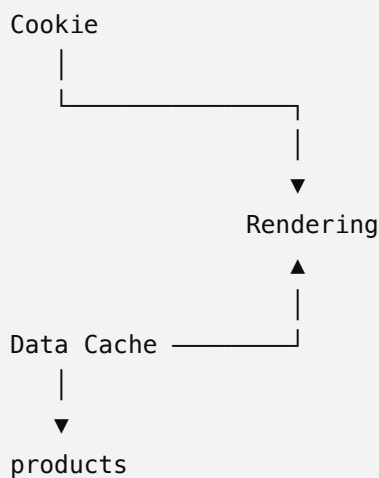
Products API data

iPhone
MacBook
AirPods



Potentially cacheable/reusable.

So conceptually:



This is a really important insight:

One route can have request-specific information AND separately cacheable data dependencies.

And this becomes **hugely important when we get to Layer 2 / Cache Components**, because modern Next.js gives you more precise control over which parts are cached and which parts remain request-dependent.



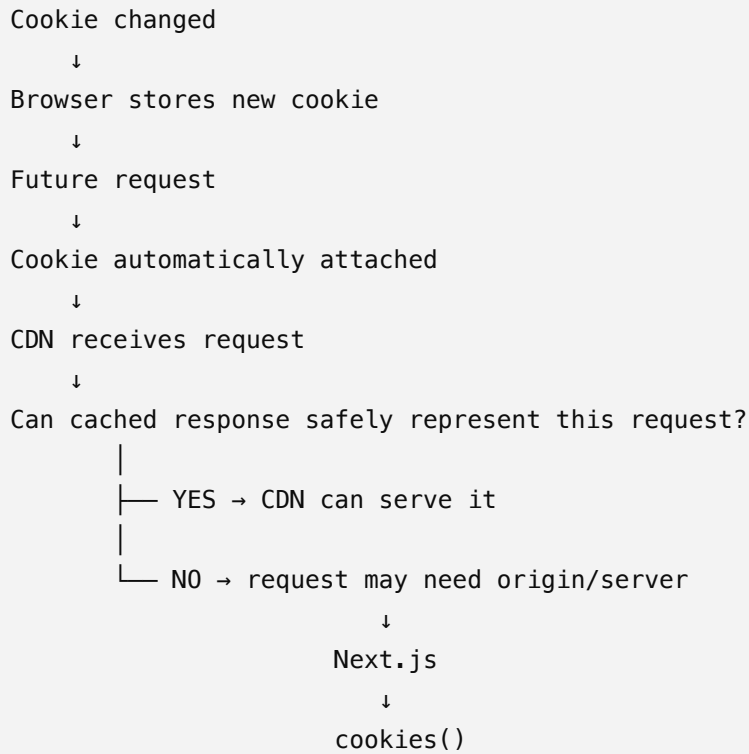
Your mental model now

Don't think:

Cookie changed
↓
Update CDN cache



Instead:



That's the whole game. 🎯

And now we're ready for the **Layer 1 debugging exercise**. I'll give you a route and code, and you tell me what happens. I'll deliberately mix **dynamic URL segments**, **cookies**, **searchParams**, **fetch**, **no-store**, **and** **revalidate** so you have to distinguish all of them.



ChatGPT can make mistakes. Check important info.